

Сервисный модуль Image.

Библиотека sbis-img для обработки изображений

Общие сведения

Библиотека предназначена для работы с графическими изображениями в различных форматах. Она является удобной обёрткой над Magick++, предоставляя более простое в использовании API и интеграцию с платформой СБИС 3.

Находится библиотека в ядре Wasaby Framework, проект **sbis-img300**.

Основные возможности

Загрузка, сохранение и преобразование форматов

Библиотека поддерживает загрузку и сохранение изображений из следующих источников:

- путь до файла;
- вектор байт;
- поток;
- RpcFile (для СБИС-3);
- Stream (для СБИС-2);
- сырые данные.

Формат автоматически определяется при загрузке и преобразуется в указанный при сохранении. Список поддерживаемых форматов приведён ниже.

Работа с цветами и цветовыми пространствами

Библиотека имеет возможность:

- Производить преобразования между цветовыми пространствами RGB, CMYK и YUV — *Image::SetColorSpace*.
- Изменять глубину цвета (количество бит на пиксель) — *Image::SetDepth*.
- Преобразовывать цветное изображение в оттенки серого (*Image::MakeGrayScale*).

Работа с геометрией

Библиотека имеет возможность:

- Изменять размеры изображения — *Image::SetHeight/SetWidth*.
- Изменять разрешение изображения (количество пикселей на дюйм) — *Image::SetDensity*.
- Отражать изображение по горизонтали и вертикали — *Image::Flip/Flop*.
- Поворачивать изображение на заданное число градусов — *Image::Rotate*.
- Вырезать область изображения — *Image::Crop*.
- Масштабировать область — *Image::Zoom*.

Работа с многостраничными изображениями

Библиотека поддерживает работу с многостраничными изображениями (формат TIFF и анимированный GIF).

Возможно разбить изображение на отдельные кадры, если есть необходимость отдельной кадровой обработки: *Image::DecomposeMultiPageImage*. Собрать изображение воедино можно методом *Image::ComposeMultiPageImage*.

Все изменяющие манипуляции с многостраничным изображением будут применяться для каждого кадра.
Все методы для получения свойств изображения будут применяться для первого кадра.

Рисование и композиция изображений

Библиотека предоставляет прямой доступ к массиву пикселей изображения и возможность гибкой композиции изображений между собой.

Поддерживаемые форматы

На данный момент поддерживаются:

- **PNG** — Portable network graphics.
- **JPEG** — Joint Photographic Experts Group format.
- **BMP** — обычный "Windows Bitmap".
- **TIFF** — Tagged Image File Format.
- **PSD** — Photoshop Document (Photoshop 8).
- **RGB** — "сырой" формат без альфа канала (без сохранения прозрачности).
- **RGBA** — "сырой" формат с альфа каналом (с сохранением прозрачности).

Не для всех форматов поддерживаются все возможности. Например, многостраничные изображения поддерживаются только для форматов TIFF и GIF.

Удаленный бэкенд обработки изображений (сервис img-remote)

У большинства развернутых в Kubernetes сервисов нет "настоящего" диска, вместо этого все файловые операции пишут в память, потребление которой для каждого pod-а ограничено. Это приводит к тому, что память заканчивается и процессы сервиса убиваются OOM-killer'ом. Удаленный бэкенд позволяет обрабатывать изображения без выделения дополнительных ресурсов pod'у локального сервиса.

Для работы с удаленным бэкендом в состав сервиса должен быть включен модуль **Image-BL**. Выбор бэкенда осуществляется при создании изображения через передачу объекта `LoadOptions`, либо глобально заданием параметра **Ядро.РаботаСИзображениями.ИспользоватьУдаленныйБэкенд**.

Изображения с разными бэкендами не могут взаимодействовать друг с другом - операции наложения изображений, сборки многостраничного изображения и т.п. закончатся ошибкой.

Удаленный бэкенд поддерживает те же операции, что и локальный за исключением обращений к отдельным пикселям(метод `ScanLine`).

Для уменьшения накладных расходов на передачу данных до сервиса, модуль **Image-BL** предоставляет функции создания изображений из HTTP GET запросов и удаленных вызовов:

- `LoadImageFromUrl` - выполняет GET запрос на заданный URL и создает **Image** из результата.

Параметры:

- `url` - запрашиваемый URL
- `options` - опции загрузки изображения
- `auth_data` - данные авторизации, используемые при запросе
- `LoadImageFromInvoke` - выполняет удаленный вызов и создает **Image** из результата. Вызываемый метод должен возвращать RPC-файл.

Параметры:

- `method` - обертка имени сервиса и метода с параметрами вызова
- `options` - опции загрузки
- `auth_data` - данные авторизации, используемые при запросе изображения

Примеры на C++:

```
1 void Example_LoadFromUrl( const String& url )
2 {
3     // Загружает изображение из результата GET запроса на url
4     Image img = img::LoadImageFromUrl(url)
```

```

5 }
6
7 void Example_LoadFromInvoke()
8 {
9     // Вызывает метод Object.GetImage на сервисе remote-service с
    параметрами (1, "test-arg") и
10     // создает изображение из результата выполнения
11     Image img = img::LoadImageFromInvoke( { "remote-service"_sv,
    "Object.GetImage", 1, L"test-arg" } );
12 }

```

Примеры на Python:

```

1 def example_loadfromurl(url):
2     # Загружает изображение из результата GET запроса на url
3     image = imgbl.LoadImageFromUrl(url)
4
5 def example_loadfrominvoke():
6     # Вызывает метод Object.GetImage на сервисе remote-service с
    параметрами (1, "test-arg") и
7     # создает изображение из результата выполнения
8     image =
    imgbl.LoadImageFromInvoke(imgbl.ServiceMethodWithArgs("remote-service",
    "Object.GetImage", 1, "test-arg"))

```

C++ API

Вспомогательные типы

В библиотеке реализовано несколько простых типов:

- **typedef ImageData** – ByteVector СБиС. Вектор байт (uint8_t). Служит для сохранения изображения в память и загрузки из памяти.
- **enum ImageFormat** – Формат изображения. Перечисление, содержащее форматы изображений.
- **enum ImageColorspace** – Цветовые пространства. Перечисление, содержащее цветовые пространства. Используется, например, для преобразования между RGB и CMYK.
- **enum CompositeOperator** — Алгоритм, применяемый для композиции двух изображений.
- **enum ObjectPosition** — Относительное положение накладываемого при композиции изображения.
- **enum ImageErrorCode** — Коды ошибок.

Классы

Pixel

Пиксель. Необходим для попиксельного просмотра или рисования изображения.

Image

Основой библиотеки является класс *Image*, находящийся в пространстве имён *sbis::img*. В нём реализована вся логика работы с изображениями. Методы этого класса, изменяющие изображение (а также конструкторы, методы загрузки и сохранения), могут выбрасывать исключение *sbis::Exception*. Большинство методов возвращает ссылку на объект, у которого они были вызваны. Полный список методов можно узнать в файле *image.hpp* в библиотеке *sbis-img*.

LoadOptions

Класс содержит набор дополнительных опций используемых при загрузке изображений:

- **mBackend** - бэкэнд для работы с изображением - локальный или удаленный;

- **mStopToken** - задает токен отмены, который будет использован для всех операций с изображением;
- **mDefaultPdfDelegate** - делегат для загрузки PDF файлов;
- **mSuggestedFormat** - предполагаемый формат файла. В некоторых случаях ImageMagick не может определить формат файла, для корректной загрузки формат можно указать в данной опции;
- **mDensity** - плотность пикселей с которой будет загружено изображение;
- **mPageRange** - диапазон страниц для загрузки;
- **mIgnoreCorruptedFrames** - игнорировать поврежденные кадры при загрузке;
- **mDisablePdfUserUnits** - отключить использование пользовательских единиц измерения при загрузке PDF;
- **mPdfAntialiasing** - включить/выключить антиалиасинг при загрузке PDF.

Функции

Получение расширения файла по его типу:

```
1 String GetExt( const ImageFormat format );
```

Константы

Базисные цвета RGB

```
1 const Pixel RED ( Pixel().MaxColorValue(), 0, 0, 0 );
2 const Pixel GREEN ( 0, Pixel().MaxColorValue(), 0, 0 );
3 const Pixel BLUE ( 0, 0, Pixel().MaxColorValue(), 0 );
```

Формат по умолчанию для сохранения изображения

```
1 const ImageFormat DEFAULT_IMAGE_FORMAT = iftPNG;
```

Примеры использования

Пример 1. Загрузка изображения (вероятно в формате TIFF), вырезание области, поворот, отражение в двух плоскостях и сохранение в файл в формате JPEG.

```
1 #include <sbis-img/image.h>
2 #include <boost/format/format_fwd.hpp>
3 #include <sbis-lib/fileu_tils.h>
4
5 void Example1( const boost::filesystem::path_t &image_path )
6 {
7     Image img1( image_path / L"img4.tif" );
8     img1.Crop( 225, 285, 305, 220 ).Rotate( 180
9     ).Flip().Flop().Flip().Save( image_path / L"test4.jpg", iftJPG );
10 }
```

Пример 2. Загрузка многостраничного изображения в формате TIFF, разбиение его на кадры и сохранение кадров в отдельных файлах (в формате по умолчанию).

```
1 #include <sbis-img/image.h>
2 #include <boost/format/format_fwd.hpp>
3 #include <sbis-lib/fileu_tils.h>
4
5 void Example2( const boost::filesystem::path_t &image_path )
```

```

6  {
7      Image img2;
8      std::vector< Image > new_images;
9
10     img2.Load( image_path / L"test5.tif" );
11     img2.DecomposeMultiPageImage( new_images );
12
13     int i = 0;
14     std::for_each( new_images.begin(), new_images.end(),
15         [&]( Image &img )
16         {
17             img.Save( image_path / ( boost::format_t( L"test_frame%1%.png"
18 ) % i++ ).str() );
19         });
20 }

```

Пример 3. Загрузка изображения, установка глубины цвета, преобразование его в оттенки серого и сохранения в формате PNG.

```

1  #include <sbis-img/image.h>
2  #include <boost/format/format_fwd.hpp>
3  #include <sbis-lib/fileu_tils.h>
4
5  void Example3( const boost::filesystem::path_t &image_path )
6  {
7      const int depth = 256;
8      Image img1;
9      img1.Load( image_path / L"img3.png" );
10     img1.SetDepth( depth ).MakeGrayScale().Save( image_path /
11         L"test3_1.png", iftPNG );
12 }

```

Python API

Класс [ImageFormat](#)

Формат изображения.

Используется в классе Image для сохранения изображения в нужных форматах.

Значения форматов:

- PNG
- JPG
- BMP
- TIFF
- PSD
- GIF
- RGBA
- RGB

Пример получения значения класса:

```

1  img.ImageFormat.PNG

```

Класс ImageColorspace

Цветовая схема.

Используется для установки цветовой схемы изображения.

Значения класса:

- CMYK
- GRAY
- RGB
- YUV
- Unknown

Пример получения значения класса:

```
1 img.ImageColorspace.RGB
```

Класс ImageErrorCode

- **COMMON_EXCEPTION** – Общая ошибка.
- **INVALID_FILE_TYPE_EXCEPTION** – Ошибка при загрузке файла недопустимого типа.
- **INVALID_FILE_SIZE_EXCEPTION** – Ошибка при загрузке файла недопустимого размера.
- **UNKNOWN_EXCEPTION** – Неизвестная ошибка.

Класс CompositeOperator

- **UndefinedComposite** – Значение не установлено.
- **OverComposite** – The result is the union of the the two image shapes with the composite image obscuring image in the region of overlap.
- **InComposite** – The result is a simply composite image cut by the shape of image. None of the image data of image is included in the result.
- **OutComposite** – The resulting image is composite image with the shape of image cut out.
- **AtopComposite** – The result is the same shape as image image, with composite image obscuring image there the image shapes overlap. Note that this differs from OverCompositeOp because the portion of composite image outside of image's shape does not appear in the result.
- **XorComposite** – The result is the image data from both composite image and image that is outside the overlap region. The overlap region will be blank.
- **PlusComposite** – The result is just the sum of the image data. Output values are cropped to 255 (no overflow). This operation is independent of the matte channels.
- **MinusComposite** – The result of composite image — image, with overflow cropped to zero. The matte chanel is ignored (set to 255, full coverage).
- **AddComposite** – The result of composite image + image, with overflow wrapping around (mod 256).
- **SubtractComposite** – The result of composite image — image, with underflow wrapping around (mod 256). The add and subtract operators can be used to perform reverible transformations.
- **DifferenceComposite** – The result of abs(composite image — image). This is useful for comparing two very similar images.
- **MultiplyComposite**
- **BumpmapComposite** – The result image shaded by composite image.
- **CopyComposite** – The resulting image is image replaced with composite image. Here the matte information is ignored.
- **CopyRedComposite** – The resulting image is the red layer in image replaced with the red layer in composite image. The other layers are copied untouched.
- **CopyGreenComposite** – The resulting image is the green layer in image replaced with the green layer in composite image. The other layers are copied untouched.
- **CopyBlueComposite** – The resulting image is the blue layer in image replaced with the blue layer in composite image. The other layers are copied untouched.

- **CopyOpacityComposite** – The resulting image is the matte layer in image replaced with the matte layer in composite image. The other layers are copied untouched.

Класс ObjectPosition

- **ForgetPosition** – Не использовать позиционирование.
- **TopLeft** – Позиционировать объект в верхний левый угол региона.
- **TopCenter** – Позиционировать объект сверху в центре региона.
- **TopRight** – Позиционировать объект в верхний правый угол региона.
- **LeftCenter** – Позиционировать объект слева по центру региона.
- **Center** – Позиционировать объект в центр региона.
- **RightCenter** – Позиционировать объект справа по центру региона.
- **LeftBottom** – Позиционировать объект в нижний левый угол региона.
- **BottomCenter** – Позиционировать объект снизу в центре региона.
- **BottomRight** – Позиционировать объект в нижний правый угол региона.

Класс Pixel

Пиксель изображения. Если пиксель получен из изображения, то он привязан к нему и его изменение вызовет изменение изображения.

Содержит следующие методы и атрибуты:

- **Pixel()** – Создать пиксель.
- **Pixel(red, green, blue, opacity)** – Создать пиксель указанного цвета.
- **Red** – Получить или установить красный компонент.
- **Green** – Получить или установить зелёный компонент.
- **Blue** – Получить или установить синий компонент.
- **Alpha** – Получить или установить прозрачность.
- **MaxColorValue** – Максимальное значение цвета. Только чтение.
- **SetRGBA** – Установить цвет пикселя.
- **GetRGBA** – Получить цвет пикселя в dict.

Класс Image

Основной класс для работы с изображениями.

Большинство методов класса возвращает текущий объект, потому что предпочтительно работать с ним, используя цепочки.

Например:

```
1 import img
2 # Создание и последующая декомпозиция в список img_list.
3 img_list = img.Image('multipage.tiff').DecomposeMultiPageImage()
4 for num, img in enumerate(img_list):
5     # Перевод каждого изображения в списке в оттенки серого, изменение
    размера (100x100),
6     # поворот на пять градусов и сохранение.
7     img.MakeGrayScale().Resize(100, 100).Rotate(5).Save('frm' +
    str(num) + '.tif')
8
9 # Создание изображения, сборка многостраничного TIFF из списка и
    сохранение.
10 img.Image().ComposeMultiPageImage(img_list).Save('new_multipage.tiff')
```

Описание класса

Конструкторы (помимо конструктора по умолчанию):

- **Image(path)** – создать изображение из файла по указанному пути.
- **Image(rpc_file)** – создать изображение из RpcFile.
- **Image(image_data)** – создать изображение из байт в памяти.
- **Image(width, height, color)** – создать пустое изображение заданного цвета.

Методы загрузки:

- **Load(path)** – загрузить изображение из файла. Метод вернёт Image, у которого он был вызван.
- **Load(image_data)** – загрузить изображение из памяти. Метод вернёт Image, у которого он был вызван.
- **Load(rpc_file)** – загрузить изображение из RPC файла. Метод вернёт Image, у которого он был вызван.
- **LoadRawBits(source, size, width, height, depth, format)** – загрузить в объект изображения Image bitmap-данные без заголовка. Метод вернёт Image, у которого он был вызван. source – массив байт (всё, что имеет блочный интерфейс). Для корректного распознавания изображения необходимо указывать ширину, высоту и глубину изображения.

Методы сохранения:

- **Save(path, image_format)** – сохранить изображение в файл в указанном формате. Метод вернёт Image, у которого он был вызван.
- **Save(rpc_file, image_format)** – сохранить изображение в RPC файл в указанном формате. Метод вернёт Image, у которого он был вызван.
- **Save(rpc_file)** – сохранить изображение в RPC файл в формате по умолчанию. Метод вернёт Image, у которого он был вызван.
- **Save(image_data, image_format)** – сохранить изображение в память в указанном формате. Метод вернёт Image, у которого он был вызван.
- **Save(image_data)** – сохранить изображение в память в формате по умолчанию. Метод вернёт Image, у которого он был вызван.

Свойства:

- **Format** – формат изображения. Нельзя поменять. Возможно только сохранить изображение в другом формате.
- **ColorSpace** – цветовая схема изображения. Тип: ImageColorspace.
- **Depth** – глубина цвета (число бит на пиксель) изображения.
- **Height** – высота изображения.
- **IsGrayscale** – если истина, палитра изображения в оттенках Серого.
- **Density** – плотность точек на дюйм.
- **Width** – ширина изображения.

Методы работы с изображением:

- **SetDensity(std::pair< size_t, size_t > const&)** – установить значение разрешения. Метод вернёт Image, у которого он был вызван.
- **Crop(from_left, from_top, slice_width, slice_height, r_width = 0, r_height = 0)** – кадрировать изображение (вырезать участок). Метод вернёт Image, у которого он был вызван.
- **Flip()** – отразить изображение по вертикали. Метод вернёт Image, у которого он был вызван.
- **Flop()** – отразить изображение по горизонтали. Метод вернёт Image, у которого он был вызван.
- **MakeGrayScale()** – перевести изображение в оттенки серого. Метод вернёт Image, у которого он был вызван.
- **Resize(new_width, new_height)** – изменить размер изображения. Метод вернёт Image, у которого он был вызван.
- **Rotate(degrees)** – повернуть изображение. Метод вернёт Image, у которого он был вызван.
- **Zoom(percents)** – изменить размер изображения на указанное количество процентов. Метод вернёт Image, у которого он был вызван.

Методы наложения изображений:

- **Composite(img, x_offset, y_offset, cop)** – наложить изображение img, используя оператор наложения cop. Смещение накладываемого изображения задаётся параметрами x_offset и y_offset.
- **Composite(img, obj_pos, cop)** – наложить изображение img, используя оператор наложения cop. Относительное положение накладываемого изображения задаётся параметром obj_pos класса ObjectPosition.

Прямой доступ к пикселям:

- **ScanLine(row_num)** – получить строку пикселей с индексом row_num, который отсчитывается от 0.
- **Image[row_num]** – доступ к строке пикселей может быть получен, если обратиться к изображению, как к списку.

- **len(Image())** – вернёт число пикселей в изображении.

Поддержка многостраничных изображений:

- **ComposeMultiPageImage(img_list)** – создать многостраничное изображение из списка объектов Image. Метод вернёт Image, у которого он был вызван.
- **DecomposeMultiPageImage(img_list)** – получить составляющие многостраничного изображения в список img_list. Метод вернёт Image, у которого он был вызван.
- **DecomposeMultiPageImage()** – получить составляющие многостраничного изображения. Метод вернёт список объектов Image.
- **DecomposeMultiPageImage(filename_pattern)** – получить составляющие многостраничного изображения в набор файлов. Параметр filename_pattern – шаблон имени файлов кадров, где %1% будет заменён на номер кадра.
- **IterateMultiPageImage(pred)** – вызвать предикат pred для каждого кадра многостраничного изображения.
- **GetFrameOfMultiPageImage(frame_number)** – получить кадр с номером frame_number.
- **GetFramesCount()** – вернёт число кадров в многостраничном изображении.

Прочее

DEFAULT_IMAGE_FORMAT – формат изображения по умолчанию. Константа для справки. Изменение ни на что не повлияет.

RED, GREEN, BLUE – пиксели соответствующих цветов. Константы.

Функции:

- **GetExt(format)** – получить расширение файла по формату.
- **Magick2Format(magick)** – получить формат по текстовой сигнатуре.

Примеры использования

Пример 1. Загрузка изображения (вероятно в формате TIFF), вырезание области, поворот, отражение в двух плоскостях и сохранение в файл в формате JPEG.

```
1 import img
2 from os import path
3
4 def example1(image_path):
5     image = img.Image(os.path.join(image_path, 'img4.tif'))
6     image.Crop(225, 285, 305, 220).Rotate(180).Flip().Flop().Flip()
7     image.Save(path.join(image_path, 'test4.jpg'),
8               img.ImageFormat.JPEG)
```

Пример 2. Загрузка многостраничного изображения в формате TIFF, разбиение его на кадры и сохранение кадров в отдельных файлах в формате по умолчанию (менее затратный по памяти, более быстрый и предпочтительный способ – это использование перегрузки **DecomposeMultiPageImage()** с шаблоном имени кадра).

```
1 import img
2 from os import path
3
4 def example2(image_path):
5     new_images = img.Image().Load(path.join(image_path,
6     'test5.tif')).DecomposeMultiPageImage()
7
8     for num, img_entry in enumerate(new_images):
9         img_entry.Save(path.join(image_path, 'test_frame' + str(num) +
10         '.tif'))
```

Пример 3. Загрузка изображения, установка глубины цвета, преобразование его в оттенки серого и сохранения в формате PNG.

```
1 import img
2 from os import path
```

```

3
4 def example3(image_path):
5     depth = 256
6     image = img.Image().Load(path.join(image_path, 'img3.png'))
7     image.SetDepth(depth).MakeGrayScale()
8     image.Save(path.join(image_path, 'test3_1.png'),
img.ImageFormat.PNG)

```

Пример 4. Рисование горизонтальной линии синего цвета, отстоящей на 100 пикселей от верхнего края.

```

1 import img
2 from os import path
3
4 def example4(image_path):
5     i = img.Image().Load(path.join(image_path, 'img4.tiff'))
6     for j in range(i.Width - 1):
7         i[j][100] = img.BLUE
8     i.Save(path.join(image_path, 'test4_1.png'), img.ImageFormat.PNG)

```

Пакетный API для преобразования изображений

Для преобразования изображений в пакетном режиме библиотека предоставляет класс TransformationBatch. Методы класса TransformationBatch повторяют методы класса Image. Преобразование изображения выполняется в вызове Image::ApplyBatch. Применение пакетного режима имеет смысл при работе с удаленным бэкендом, т.к. позволяет провести серию преобразований за один удаленный вызов.

```

1 #include <sbis-img/image.h>
2 #include <sbis-img/image_transformation_batch.hpp>
3
4 void Example5( const String& image_path )
5 {
6     Image image{ image_path };
7     TransformationBatch batch;
8     batch.Resize( 200, 100 ).Rotate( 90 );
9     image.ApplyBatch( batch );
10 }

```

```

1 def example5(image_path):
2     i = img.Image(image_path)
3     batch = img.TransformationBatch()
4     batch.Resize(200, 100).Rotate(90)
5     i.ApplyBatch(batch)

```